

RUST CRATE CARGO CACHE COMPROMISE HUNT

Retrospective hunt for officially listed
compromised and attacker-owned crate
artifacts on developer and CI systems.

HUNTING QUERY

CONFIDENCE: MEDIUM

SOURCE-CONFIRMED BEHAVIOR

retain facts, isolate interpretation

DeviceFileEvents

direct telemetry boundary

HUNT · VALIDATE · TUNE

no automatic malicious verdict

REPORT DATE

23 August 2026

VERSION

v1.0

PRIMARY TELEMETRY

Microsoft Defender XDR ·
DeviceFileEvents

ATT&CK

T1195.001

Evidence-led dossier. Query status remains an untested
implementation sketch until executed in the target environment.

INDEPENDENT RESEARCH FORMAT · CONTROLLED UI/UX MASTER



EXECUTIVE ASSESSMENT

Decision-grade summary for detection engineering and SOC operations.

ASSESSMENT

EXECUTIVE SUMMARY

Use exact Cargo cache artifacts for high-signal scoping, not as proof of payload execution.

The Rust Security Response Team published exact compromised versions and attacker-owned crate names to search under Cargo registry caches. Microsoft Defender for Endpoint can record file-system activity in DeviceFileEvents, making cache artifacts a direct retrospective pivot when developer workstations and persistent CI runners are onboarded.

OPERATIONAL VALUE

Fast exposure scoping across systems that may retain poisoned dependencies.

The hunt identifies official package artifacts that can persist after crates were removed from crates.io. A match supports dependency and build investigation, but does not establish that build.rs executed or that a second-stage payload ran.

9.1

01

OPERATIONAL VALUE



Medium

02

DETECTION CONFIDENCE



Hunting

03

OPERATIONAL READINESS



SOURCE-CONFIRMED FACTS

Observed behavior

- Rust confirmed that proc-macro1 contained a build script which downloaded a malicious payload.
- The compromised releases were arrayref 0.3.10, interment 0.8.7 and append-only-vec 0.1.9.
- Rust explicitly recommends checking Cargo caches for those files and six attacker-owned crate-name prefixes.

ANALYTICAL ASSESSMENT

What the evidence supports

- Cargo package resolution can place the exact .crate archive in a local registry cache.
- A build using a poisoned dependency can execute the malicious build script even when the library API is never called.
- Deleted crates can remain in warm developer, runner or mirror caches after registry removal.

DETECTION ASSUMPTIONS

What must be validated

- Microsoft Defender for Endpoint covers relevant developer systems and persistent or self-hosted runners.
- DeviceFileEvents retains the cache-file activity within available telemetry retention.
- Cargo used a conventional registry cache path containing .cargo, registry and cache.

Confidence boundary

The primary query uses only filenames officially published by the Rust Security Response Team. Network indicators and stage-two artifacts from third-party reporting are excluded. A cache match is exposure evidence, not execution proof.



THREAT MECHANICS

Source-observed chain and explicit observability boundaries.

ATTACK FLOW

01

Compromise publishing access

The legitimate crate owner account or machine is believed to have been compromised.

02

Publish poisoned versions

Three legitimate packages were released with malicious dependency changes.

03

Resolve dependency

Cargo downloads the affected archive into a registry cache.

04

Run build script

Building a resolved malicious version can execute `proc-macro1 build.rs`.

05

Fetch payload

The source-analyzed script downloads and launches a platform-specific payload.

Observable core

Exact official `.crate` filenames under a Cargo registry cache, together with device, time, file origin and initiating-process context when populated.

Non-observable boundary

Dependency resolution intent, build-script execution, payload success, downstream compromise and secret theft from the cache event alone.

DETECTION HYPOTHESIS

Officially listed package artifacts should remain visible in Cargo registry caches.

If a developer or CI system downloaded an affected or attacker-owned crate, endpoint file telemetry should show the official archive name under a Cargo registry cache. The event can be distinguished from legitimate package activity by exact source-published names, repository ownership, lockfile state, build timing and authorized security-test context.

REQUIRED TELEMETRY

DeviceFileEvents

- DeviceFileEvents from developer workstations and persistent CI runners.
- FileName and FolderPath.
- Timestamp and stable device identity.

OPTIONAL ENRICHMENT

Context that improves disposition

- Cargo.lock, repository and dependency graph.
- CI job, runner and source-control metadata.
- File origin, hash and initiating-process context.

UNAVAILABLE HERE

Do not infer

- Proof that `build.rs` executed from a cache-file event.
- Payload execution outcome.
- Ephemeral runners destroyed before endpoint collection.



TELEMETRY & EVIDENCE MODEL

Minimum data contract and quality gates for a defensible hunt.

DATA CONTRACT

PRIMARY DATA SOURCE

Microsoft Defender XDR - DeviceFileEvents

Microsoft documents DeviceFileEvents as file creation, modification and other file-system events, including file path, hashes, origin and initiating-process context.

EVIDENCE BOUNDARY

A matching event is a lead, not a verdict.

The query does not filter ActionType because the documented field values depend on the in-portal schema. Optional enrichment may be empty and is never required for a match.

REQUIRED AND ENRICHMENT FIELD MODEL

CATEGORY	FIELD	PURPOSE	REQUIREMENT
Log event	DeviceFileEvents	Endpoint file-system activity.	REQUIRED
Artifact	FileName	Official package archive names.	REQUIRED
Location	FolderPath	Cargo registry cache discriminator.	REQUIRED
Time	Timestamp	Exposure and build-window pivot.	REQUIRED
Device	DeviceId, DeviceName	System ownership and correlation.	REQUIRED
Integrity	SHA1, SHA256, FileSize	Optional sample triage.	OPTIONAL
Origin	FileOriginUrl, FileOriginIP	Optional acquisition context.	OPTIONAL
Process	InitiatingProcess*	Optional downloader and account context.	OPTIONAL

Data quality requirements

- Confirm MDE coverage on developer endpoints and self-hosted runners.
- Download a benign crate in an isolated onboarded system and verify cache events.
- Measure FolderPath and FileName completeness across operating systems.
- Check whether affected build dates remain inside telemetry retention.

Known failure conditions

- Developer and CI assets are not onboarded.
- Ephemeral runners were destroyed before collection.
- File events predate retention or the Cargo cache is excluded.
- Dependencies were vendored or mirrored outside the expected path.



DETECTION LOGIC

Primary-platform logic with every condition traceable to evidence.

HUNTING QUERY

DETECTION OBJECTIVE

Find official compromised or attacker-owned crate archives in Cargo caches.

Scope potentially exposed devices and pivot into lockfiles, repositories, builds, secrets and process evidence.

ANALYTICAL LOGIC

Direct filters and investigable output.

Three exact compromised-version filenames plus six source-published attacker-owned prefixes, restricted to Cargo registry cache paths. No threshold, rarity, join, hash list or execution verdict.

KQL · UNTESTED HUNTING

```
// Hunting query - Untested implementation sketch
// Primary platform: Microsoft Defender XDR / Advanced Hunting
// Required telemetry: Microsoft Defender for Endpoint file events on developer
// workstations and persistent or self-hosted CI runners.
// Evidence: the Rust Security Response Team recommends checking Cargo registry
// caches for these exact compromised versions and attacker-owned crate names.
DeviceFileEvents
| where FolderPath contains ".cargo"
    and FolderPath contains "registry"
    and FolderPath contains "cache"
| where
    FileName in~ (
        "arrayref-0.3.10.crate",
        "internment-0.8.7.crate",
        "append-only-vec-0.1.9.crate"
    )
    or (
        FileName endswith ".crate"
        and (
            FileName startswith "proc-macro1-"
            or FileName startswith "proc-macro-en-"
            or FileName startswith "aovine-"
            or FileName startswith "arone-"
            or FileName startswith "aronenao-"
            or FileName startswith "tinymember-"
        )
    )
| project
    Timestamp, DeviceId, DeviceName, ReportId, ActionType,
    FileName, FolderPath, PreviousFileName, PreviousFolderPath,
    FileSize, SHA1, SHA256, FileOriginUrl, FileOriginIP,
    InitiatingProcessAccountDomain, InitiatingProcessAccountName,
    InitiatingProcessAccountUpn, InitiatingProcessFileName,
    InitiatingProcessFolderPath, InitiatingProcessCommandLine,
    InitiatingProcessId, InitiatingProcessUniqueId
| order by Timestamp desc
```

STATUS	THRESHOLD	JOIN / SEQUENCE	PRIMARY KEY	DISPOSITION
Untested hunting	None	None	DeviceId + FolderPath + FileName + Timestamp	Manual triage

What a match means

An officially listed archive name was observed in a Cargo registry cache and requires dependency/build investigation.

What a match does not prove

Malicious build execution, payload launch, credential theft, persistence or attacker attribution.



TRIAGE & INVESTIGATION

Evidence collection before escalation.

SOC WORKFLOW

NUMBERED WORKFLOW

- 01

Preserve the file event.
Capture device, path, time, origin, hashes and initiating-process context.
- 02

Identify repository scope.
Locate Cargo.lock, manifests, dependency graph and owning team.
- 03

Review build history.
Find jobs and developer builds resolving the affected version.
- 04

Inspect execution evidence.
Search for build.rs, rustc/cargo child activity and source-described payload launch.
- 05

Rotate exposed secrets.
If a malicious build ran, revoke CI, registry, cloud and developer credentials in scope.

Escalation gate

Escalate when an exact artifact is tied to an affected lockfile or build, the system held sensitive credentials, or process/network evidence supports build-script or payload execution.

Primary benign overlap

Authorized security testing, incident-response collections, internal mirror restoration and deliberately retained samples can create the same filenames. Tune only with owned test scope, repository context and documented retention purpose.

INVESTIGATION PIVOTS

Device
Owner, operating system, MDE health and retention.

Repository
Cargo.lock, manifest and transitive dependency path.

Build
Job ID, runner, timestamps and artifact provenance.

Process
cargo/rustc lineage and child execution.

Secrets
Tokens available to the build or developer session.

Network
Source-described download destinations and follow-on traffic.

DISPOSITION CRITERIA

OUTCOME	MINIMUM EVIDENCE	ACTION
BENIGN	Approved, owned and repeatable activity.	Document and tune narrowly.
SUSPICIOUS	Unknown activity without corroborated malicious follow-on.	Deepen endpoint review.
LIKELY MALICIOUS	Unauthorized match plus coherent file, process, network or control-plane evidence.	Escalate incident response.



FALSE POSITIVES & VALIDATION

Operational controls required before promotion.

QUALITY GATE

FALSE-POSITIVE MATRIX

SCENARIO	WHY IT OVERLAPS	TUNING ACTION
Security validation	A lab may intentionally fetch the affected archive.	Require approved test host, operator and window.
Incident-response collection	Responders may copy a sample into a Cargo-like path.	Validate case ownership and acquisition provenance.
Internal mirror or cache restore	Backup recovery may reintroduce an archive without a build.	Confirm mirror owner, restore event and absence of build execution.

Baseline plan

- Run across the maximum available file-event retention.
- Review every match with repository and CI owners.
- Measure cache-path coverage on Windows, Linux and macOS.
- Do not introduce a count or rarity threshold.

Validation procedure

- Verify a controlled cache write produces DeviceFileEvents.
- Execute the positive and negative filename/path controls.
- Confirm projected fields and optional enrichment behavior.
- Compare matches against Cargo.lock and CI build history.

TEST CASES

CASE	EXPECTED RESULT
Download a benign crate into a standard Cargo cache	Telemetry validation only; query must not match.
Place arrayref-0.3.10.crate in an isolated standard cache	Expected match for exact compromised version.
Place an unrelated arrayref version in the same cache	No match.
Place the affected filename outside a Cargo cache path	No match by design.

Hunting

Initial query

Candidate

Current status

Production

Measured precision

Optimized

Stable feedback

Success

Controlled positive artifacts match, negative controls do not, path and device fields are complete, and historical matches can be assigned to repositories and builds.

Rejection

Relevant assets lack durable endpoint coverage or exact cache filenames cannot be retained with stable device and path context.

Failure

Controlled cache writes produce no usable DeviceFileEvents record or affected activity is outside retention.



ATT&CK, CONCLUSION & REFERENCES

Coverage statement, residual risk and document control.

CLOSURE

MITRE ATT&CK MAPPING

TECHNIQUE	NAME	COVERAGE STATEMENT
T1195.001	Compromise Software Dependencies and Development Tools	Directly covers compromised package dependencies distributed through a software package registry and resolved by developer or build tooling.

Detection coverage summary

Strong for the officially listed archives retained in conventional Cargo caches; weak for cleared caches, vendored dependencies, private mirrors, renamed files and unmonitored ephemeral runners.

Residual risk

The actor can change crate names or versions, use another ecosystem, erase caches or compromise builds without endpoint retention. Dependency inventory, lockfile review and CI hardening remain mandatory.

ANALYST CONCLUSION

A precise retrospective hunt with a strict evidence boundary.

Official Rust guidance supplies exact artifacts and Microsoft documents the endpoint fields. The signal is specific and operationally useful for scoping, but coverage and retention vary and a cache file does not prove execution, so Medium-confidence Hunting is the defensible classification.

REFERENCES

- Rust Security Response Team, "Supply-chain attack on arrayref," 20 August 2026. <https://blog.rust-lang.org/2026/08/20/supply-chain-attack-on-arrayref/>
- StepSecurity, "arrayref Rust crate supply chain attack," accessed 23 August 2026. <https://www.stepsecurity.io/blog/arrayref-rust-crate-supply-chain-attack>
- Microsoft Learn, "DeviceFileEvents table in the advanced hunting schema," accessed 23 August 2026. <https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-devicefileevents-table>
- MITRE ATT&CK, T1195.001, Compromise Software Dependencies and Development Tools; accessed 23 August 2026. <https://attack.mitre.org/techniques/T1195/001/>

DOCUMENT CONTROL

VERSION	DATE	STATUS	OWNER
v1.0	23 August 2026	Hunting	Detection Engineering

Publication control

Eight A4 pages. URLs are defanged. No customer data, credentials, HTML source or rendering assets are included in the public repository.